

사내 실습 워크숍 · 2시간 속성

매일 하는 귀찮은 크롤링 업무, AI로 없애는 가장 쉬운 방법

- 1 Claude Code(두뇌)와 Playwright(손가락)를 결합해, 코딩 지식 없이 자연어 한 마디로 JS 렌더링 사이트까지 크롤링하는 방법을 배웁니다.
- 2 경쟁사 공개방 수, 앱 버전 변경 등 매일 손으로 확인하던 반복 데이터 수집을 AI가 자동으로 Google Sheets에 누적해줍니다.
- 3 수집된 결과를 Google Chat으로 자동 전송하는 알림까지 연결해, 아침에 앱을 열지 않아도 변화를 즉시 파악할 수 있는 완전한 모니터링 자동화 파이프라인을 직접 만들어봅니다.

Claude Code

Playwright

Google Sheets

코딩 지식 불필요

오늘의 흐름

이 순서대로 진행합니다



오늘 끝나면 — 크롤링이 매일 자동으로 실행되고 Google Chat으로 결과가 도착합니다

개념

크롤링, API, Playwright

데이터를 어떻게 가져오는지,
언제 크롤링을 쓰고 언제 API를 쓰는지
먼저 이해하고 실습으로 들어갑니다.

개념

개념 · 크롤링이란?

크롤링 = 웹사이트에서 데이터를 자동으로 가져오는 것

📖 크롤링이란?

사람이 브라우저로 사이트에 접속해서
숫자나 텍스트를 눈으로 확인하는 과정을
코드가 대신 자동으로 하는 것입니다.

사람이 매일 하는 것

브라우저 열기 → 사이트 접속 → 숫자 확인

크롤링이 대신 하는 것

위 과정을 코드가 자동으로 반복

💡 활용 예시

- 경쟁사 공개방 개수 — 매일 자동 수집
- 앱 버전 변경 — 업데이트 즉시 감지
- 가격 · 리뷰 · 공지사항 모니터링

🔄 자동화하면 이렇게 됩니다

- 1 매일 같은 사이트를 열 필요 없음
코드가 접속하고 데이터를 가져옴
- 2 결과가 자동으로 시트에 쌓임
날짜별 누적 → 트렌드 파악 가능
- 3 변화가 생기면 즉시 알림
Google Chat으로 자동 전송

개념 · 크롤링 VS API

크롤링 vs API — 언제 각각 써야 하는가

API가 있으면 API가 정답입니다. 크롤링은 API가 없을 때 쓰는 방법입니다.

✅ 공개 API 사용

1순위

서비스가 공식 API를 제공하면
JSON을 직접 받아옴.
가장 안정적, 서버 부담 없음.

```
# 예: iOS App Store  
itunes.apple.com/lookup  
→ JSON 응답 즉시 파싱
```

🔵 requests 사용

2순위

API는 없지만 HTML에
데이터가 그대로 있을 때.
가볍고 빠름.

```
# 정적 HTML 사이트  
requests.get(url)  
→ BeautifulSoup 파싱
```

⚡ Playwright 사용

마지막 수단

JS가 데이터를 그리는
React/Vue 앱.
API도 없고 정적 HTML도 아닐 때만.

```
# 예: 삼성갤럭시, Google Play  
browser.launch()  
→ JS 실행 후 파싱
```



Claude가 판단합니다 — URL만 알려주면 Claude가 어떤 방법이 맞는지 자동으로 선택해줍니다

개념 · 크롤링 원칙

크롤링하기 전, 반드시 확인하세요

⚠️ 이걸 하면 안 됩니다

- 로그인해야만 보이는 회원 전용 데이터 수집
- 개인정보 (이름·연락처·주소) 수집·저장
- 짧은 간격으로 반복 요청해 서버 과부하 유발
- 이용약관에서 크롤링을 명시 금지한 사이트
- 수집 데이터를 외부 배포·상업적 활용

✅ 우리가 하는 것

- 비로그인 공개 페이지의 통계 숫자만 수집
- 하루 1회 실행, 서버 부담 최소화
- 방 개수 앱 버전 등 집계 데이터만 내부 활용

📄 robots.txt 확인

사이트의 크롤링 허용 여부는

사이트주소/robots.txt 에서 확인합니다.

```
Disallow: / → 크롤링 차단  
Allow: /guest/ → 공개 허용
```

🕒 요청 간격 원칙

요청 사이에 **1~3초 딜레이**를 넣으세요.

빠를수록 IP 차단 위험이 높아집니다.

Claude에게 **"요청 사이에 2초 기다려줘"**라고 하면 됩니다.

개념 · PLAYWRIGHT란?

왜 일반 방법으론 안 되고 Playwright인가

✗ 일반 크롤러 (requests)

서버에서 받은 HTML 파일만 읽어옴.
JavaScript는 실행하지 않음.

```
# 삼성오픈투는 React 앱 → JS가 숫자를 그림  
방 개수: 0 → JS 미실행 상태
```

✓ Playwright

실제 Chrome 브라우저를 뒤에서 실행.
JS 렌더링이 완료된 후 데이터를 읽음.

```
# JS 실행 완료 후 → 실제 숫자  
방 개수: 4,872 → 정확한 값
```

Playwright가 할 수 있는 것

실제 브라우저 조종 로그인·클릭·스크롤 JS 렌더링 대기 팝업 처리

오늘 사용하는 사이트

삼성오픈투 — React 앱 Google Play — JS 렌더링 iOS — API (Playwright 불필요)

STEP 01

Claude 데스크탑 앱 설치

Windows에 설치하고 로그인합니다.
이게 오늘 모든 작업의 시작입니다.

STEP 01 · 설치

Claude 데스크탑 앱 설치 & 실행

1 공식 사이트 접속

`claude.ai/download` → Windows(.exe) 다운로드

2 설치 실행

안내에 따라 설치 완료

3 앱 실행 → 로그인

Claude 계정으로 로그인

4 Claude Code 화면 진입

명령을 입력할 수 있는 터미널 화면 확인

✅ **체크포인트** — 앱 실행 · 로그인 완료 · Claude Code 화면 진입

⚠️ 사전 준비

설치 파일은 강의 전 미리
다운로드해 주세요.

당일에는 **"로그인 후 실행"**부터 바로 시작합니다.

📁 실제 실행 화면

(스크린샷 — 당일 교체)

STEP 02

작업 폴더 + Google Sheets 준비

Claude가 작업할 공간과
수집 데이터를 쌓을 시트를 먼저 만듭니다

STEP 02 · 작업 폴더

작업 폴더 생성

1 최상위 프로젝트 폴더 생성

예: C:\Users\이름\competitor-monitor

이 폴더만 직접 만들면 됩니다

2 Claude Code에서 폴더 열기

Claude Code → 폴더 선택 → 작업 폴더 지정

✅ 내부 구조는 Claude가 알아서

`src/`, `.env`, `requirements.txt` 등

폴더 내부 파일과 구조는

Claude Code가 자동으로 생성합니다.

직접 만들 필요 없습니다.

📁 최종 폴더 구조 예시

```
competitor-monitor/  
├── src/ ← Claude가 생성  
│   └── collect.py  
├── .env ← Claude가 생성  
├── service-account.json  
├── 내가 직접 넣을  
└── requirements.txt
```

STEP 02 · GOOGLE SHEETS

수집 데이터를 쌓을 시트 만들기

1 새 스프레드시트 생성

`sheets.new` 접속 → 새 시트 생성

2 탭 이름 설정

하단 + 버튼으로 탭 추가 → 아래 이름으로 설정

탭 이름

`raw_supply_snapshots``app_versions`

저장 데이터

방 개수 (실습 ①)

앱 버전 (실습 ②)

 스프레드시트 링크 복사해두기

```
docs.google.com/spreadsheets  
/d/여기가 Sheet ID/edit
```

이 URL 전체를 복사해주세요.
Claude에게 요청할 때 바로 붙여넣습니다.

 왜 탭 이름이 중요한가?

Claude가 코드를 작성할 때 탭 이름을 기준으로 데이터를 저장합니다. 미리 만들어줘야 오류 없이 기록됩니다.

STEP 03

Claude에게 첫 요청하기

자연어로 요청하면 Claude가 코드를 짭니다.
이 과정에서 Playwright가 자동으로 설치됩니다.

STEP 03 · 요청 방법

Claude에게 요청할 때 이 4가지를 담으세요

프롬프트가 막막할 때, 이 4가지 항목을 채우면 Claude가 정확하게 코드를 만들어줍니다

1 어떤 링크에서 가져올지

크롤링할 URL을 정확히 알려줍니다

예: "이 링크에서 가져와줘 → `https://web.33m2.co.kr/guest/room`."

2 그 정보가 페이지의 어디에 있는지

숫자-텍스트가 화면 어느 부분에 보이는지 설명합니다

예: "방 목록 위쪽에 타일별로 카드가 있고, 각 카드 오른쪽에 개수 숫자가 있어"

3 이 데이터가 매일 바뀌는 건지

수집 주기와 누적 여부를 알려줍니다

예: "이 숫자는 매일 달라져. 날짜와 함께 사이트에 한 행씩 추가해줘"

4 결과를 어디에 저장할지 (시트 링크)

아까 만든 Google Sheets URL과 탭 이름을 직접 붙여넣습니다

예: "이 시트에 저장해줘 → `[스프레드시트 URL]` · 탭 이름: `raw_supply_snapshots`"

STEP 03 · 자동 설치

요청하면 **Playwright**가 자동으로 설치됩니다

CLAUDE CODE 터미널에서 일어나는 일

```
# 1. Claude가 사이트를 분석  
→ JS 렌더링 필요한 사이트 감지  
# 2. 필요한 도구 자동 설치  
$ python -m venv .venv  
$ pip install playwright gspread  
$ playwright install chromium  
✓ 환경 설정 완료  
# 3. 크롤링 코드 생성 후 실행  
$ python collect.py
```

✓ 내가 할 일

4가지 context를 담아서 요청하기

설치환경구성·코드 작성·테스트는 **Claude**가 전부 처리**왜 Playwright를 선택했는가?**

Claude가 URL을 보고 해당 사이트가 JavaScript로 렌더링된다는 것을 판단 → 일반 크롤러로는 빈값이 나오므로 Playwright를 자동 선택

STEP 04

Google Sheets 서비스 계정 연결

Claude가 Google Sheets에 직접 데이터를 쓸 수 있도록

인증 열쇠(서비스 계정)를 연결합니다

STEP 04 · 서비스 계정이란?

왜 서비스 계정이 필요한가

 서비스 계정 = 프로그램 전용 구글 계정

사람이 로그인하지 않아도
코드가 Google Sheets에 자동으로
데이터를 쓸 수 있게 해주는 열쇠

 이게 있으면

매일 새벽에 자동으로 실행되는 코드가
내 시트에 데이터를 쓸 수 있음
(내가 컴퓨터 앞에 없어도)

 이게 없으면

코드가 "권한 없음" 오류 발생
Sheets에 아무것도 기록 불가

전체 연결 구조

내 코드 (Claude가 작성)

↓ service-account.json 사용

Google Sheets API

↓ 편집 권한 확인

raw_supply_snapshots 탭에 기록

 JSON 파일 = 열쇠

비밀번호와 동일합니다.

GitHub에 올리거나 공유하지 마세요.

작업 폴더 안에만 보관.

STEP 04 · JSON 발급 & 배치

서비스 계정 만들고 폴더에 넣기

1 Google Cloud Console 접속

`console.cloud.google.com` → 프로젝트 선택/생성

2 Sheets API 활성화

API 및 서비스 → 라이브러리 → "Google Sheets API" → 사용 설정

3 서비스 계정 생성

IAM → 서비스 계정 → + 만들기 → 이름 입력 → 완료

4 JSON 키 다운로드

서비스 계정 클릭 → 키 탭 → 키 추가 → JSON → 다운로드

5 작업 폴더에 파일 넣기

다운로드한 `service-account.json` 을 작업 폴더 최상단에 복사

✓ 파일 넣은 후 Claude에게

service-account.json 파일을
작업 폴더에 넣었어.
이걸 사용해서 Google Sheets에
연결해줘.

서비스 계정 이메일 형태

`이름@프로젝트ID`
`.iam.gserviceaccount.com`

이 이메일을 다음 단계에서 사용

STEP 04 · 시트 권한

Google Sheets에 서비스 계정 편집 권한 추가

1 아까 만든 Google Sheets 열기

2 우측 상단 공유 버튼 클릭

3 서비스 계정 이메일 입력

`이름@프로젝트.iam.gserviceaccount.com`

4 권한: 편집자 선택 → 완료

✓ 체크포인트 — JSON 파일 폴더에 있음 · 시트 공유 완료

📦 왜 이 작업이 필요한가?

Google Sheets는 기본적으로 아무나 수정할 수 없습니다.

서비스 계정 이메일을 공유하면 코드가 "이 계정으로 로그인"해서 데이터를 쓸 수 있게 됩니다.

마치 공유 문서에 로봇 계정을 초대하는 것과 같습니다.

실습 ①

삼삼엠투 방 개수 수집

어떤 링크에서, 어떤 정보를, 얼마나 자주 가져올지
Claude에게 정확히 알려주는 방법을 실습합니다

실습 ① · CONTEXT 구성

Claude에게 전달할 4가지 정보

1 수집할 링크

전국: `https://web.33m2.co.kr/guest/room`서울: `https://web.33m2.co.kr/guest/room?keyword=서울`

2 그 정보가 어디에 있는지

"방 목록 상단에 오피스텔·모텔, 아파트·빌라, 단독·투룸 등 타입별 카드가 있고, 각 카드에 개수 숫자가 표시돼 있어. 이 숫자들의 합계를 가져와줘."

3 이 데이터가 매일 바뀌는 건지

"이 숫자는 매일 달라져. 오늘 날짜와 함께 시트에 한 행씩 누적해서 추가해줘. 덮어쓰지 말고."

4 어디에 저장할지 (링크 직접 붙여넣기)

"이 Google Sheets에 저장해줘 → {스프레드시트 URL}"

탭 이름: `raw_supply_snapshots` "

실습 ① · 결과

실행하면 시트에 이렇게 쌓입니다

터미널 출력

```
$ python collect.py
[INFO] 삼성행투 전국 페이지 로딩...
[INFO] JS 렌더링 대기 중...
[INFO] 방 개수 파싱 완료
[OK] 전국: 4,872개
[OK] 서울: 1,203개
✓ Sheets 저장 완료
```

시트에 쌓이는 데이터

DATE	REGION	COUNT
2026-06-03	nationwide	4,857
2026-06-04	nationwide	4,872
2026-06-04	seoul	1,203

오류가 나면

오류 메시지 전체를 복사해서
Claude에게 붙여넣기 →
"이 오류 고쳐줘" 한 마디

실습 ②

앱 버전 자동 감지

iOS · Android 앱 버전이 바뀌면
자동으로 감지하고 사이트에 기록합니다

실습 ② · CONTEXT 구성

iOS / Android — Claude에게 전달할 정보

iOS App Store

1 수집 링크

iTunes API (인증 불필요)

`itunes.apple.com/lookup?id=앱ID&country=kr`

2 정보 위치

API가 JSON으로 응답함.

응답 안의 `version`, `currentVersionReleaseDate`, `releaseNotes` 필드

3 변경 여부 판단

"어제 기록된 버전과 오늘 버전이 다르면 `is_new_version=TRUE`로 표시해줘"

4 저장 위치

[스프레드시트 URL]

탭: `app_versions`

Android Google Play

1 수집 링크

`play.google.com/store/apps/details``?id=kr.co.33m2.android`

2 정보 위치

페이지 중간 "업데이트 날짜" 섹션에
버전 번호와 릴리즈 노트가 있어

3 변경 여부 판단

iOS와 동일하게 전날과 비교해서

`is_new_version` 표시

4 저장 위치

동일 시트, 탭: `app_versions`

실습 ② · 결과

버전 변경 감지 결과

시트에 쌓이는 데이터

```
date 2026-06-04
platform ios
version 3.2.1 ← 어제: 3.2.0
is_new_version TRUE
change: 3.2.0 → 3.2.1
date 2026-06-04
platform android
version 3.2.1 ← 변화 없음
is_new_version FALSE
```

앱 ID / 패키지명 찾는 방법

iOS

App Store URL의 `id` 뒤 숫자 AndroidPlay Store URL의 `id=` 뒤 값

앱 설명 변경도 감지

"어제와 오늘 앱 설명 텍스트가 달라지면 is_description_changed=TRUE로 표시해줘"
라고 요청하면 됩니다

✓ 체크포인트



실습 ① 시트에 방 개수 기록됨



실습 ② 시트에 앱 버전 기록됨

마무리

Google Chat으로 알림 받기

수집 결과를 매일 Google Chat으로 자동 전송합니다.

웹훅 URL 설정 방법을 알아봅니다.

마무리 · 웹훅 설정

Google Chat 웹훅 URL 만들기

1 알림 받을 Google Chat 스페이스 열기

채팅 스페이스 또는 개인 알림용 스페이스 선택

2 앱 및 통합 메뉴 열기

스페이스 이름 클릭 → 앱 및 통합 → 수신 웹훅

3 수신 웹훅 추가

이름 입력 (예: "모니터링 봇") → 저장

4 URL 복사 → Claude에게 전달

생성된 URL을 복사해서 .env에 저장하거나 Claude에게 직접 알려줌

```
# .env에 추가
GOOGLE_CHAT_WEBHOOK_URL
https://chat.googleapis.com/v1/spaces/...
```

Claude에게 이렇게 요청

.env의 GOOGLE_CHAT_WEBHOOK_URL로 수집 결과 요약물 한국어로 정리해서 Google Chat에 전송하는 함수를 만들어줘.

방 개수 변화 + 앱 버전 변경 내용을 포함해줘.

⚠ 웹훅 URL 보안

웹훅 URL은 비밀번호와 같습니다.

.env에만 저장하고 코드에 직접 쓰지 마세요.

마무리 · 완성

매일 오전 9시, 이런 메시지가 옵니다

GOOGLE CHAT 수신 메시지

Daily 경쟁사 모니터링 · 2026-06-04

[공개방 수]

전일 4,872

전일 +15 / 7일전 +42

전일 1,203

전일 +3 / 7일전 +11

[앱 업데이트]

iOS

3.2.0 → 3.2.1

이전 버전은 더 이상 지원하지 않습니다. 업데이트를 권장합니다.

✓ 전체 자동화 완성 구조

① Playwright 크롤링



② Google Sheets 저장



③ Google Chat 전송

🔄 자동 실행 등록

Claude에게

"매일 오전 9시에 자동 실행되게

스케줄러 등록해줘"

라고 하면 완성

감사합니다

Q & A

내 업무 중 자동화하고 싶은 크롤링이 있으신가요?
아이디어 공유 및 질문 받겠습니다.

두뇌
Claude Code
코드 작성·수정

손가락
Playwright
브라우저 조종

결과
Sheets + Chat
자동 기록·알림